

习题十

一、选择题

1. 某内排序方法的稳定性是指()。
A. 该排序算法不允许有相同的关键字记录 B. 该排序算法允许有相同的关键字记录
C. 平均时间为 $O(n \log n)$ 的排序方法 D. 以上都不对
2. 下面给出的四种排序法中()排序法是不稳定性排序法。
A. 插入 B. 冒泡 C. 二路归并 D. 堆积
3. 若需在 $O(n \log_2 n)$ 的时间内完成对数组的排序,且要求排序是稳定的,则可选的排序方法是 ()
A. 快速排序 B. 堆排序 C. 归并排序 D. 直接插入排序
4. 下面给出的四种排序方法中, 排序过程中的比较次数与排序方法无关的是。()
A. 选择排序法 B. 插入排序法 C. 快速排序法 D. 堆积排序法
5. 对一组数据 (84, 47, 25, 15, 21) 排序, 数据的排列次序在排序的过程中的变化为 (1) 84 47 25 15 21
(2) 15 47 25 84 21 (3) 15 21 25 84 47 (4) 15 21 25 47 84 则采用的排序是 ()。
A. 选择 B. 冒泡 C. 快速 D. 插入
6. 对序列{15, 9, 7, 8, 20, -1, 4}进行排序, 进行一趟后数据的排列变为{4, 9, -1, 8, 20, 7, 15}; 则采用的是 () 排序。
A. 选择 B. 快速 C. 希尔 D. 冒泡
7. 若上题的数据经一趟排序后的排列为{9, 15, 7, 8, 20, -1, 4}, 则采用的是 () 排序。
A. 选择 B. 堆 C. 直接插入 D. 冒泡
8. 下列排序算法中()不能保证每趟排序至少能将一个元素放到其最终的位置上。
A. 快速排序 B. shell 排序 C. 堆排序 D. 冒泡排序
9. 一组记录的关键码为 (46, 79, 56, 38, 40, 84), 则利用快速排序的方法, 以第一个记录为基准得到的一次划分结果为 ()。
A. (38,40,46,56,79,84) B. (40,38,46,79,56,84) C. (40,38,46,56,79,84) D. (40,38,46,84,56,79)
10. 在下面的排序方法中, 辅助空间为 $O(n)$ 的是()。
A. 希尔排序 B. 堆排序 C. 选择排序 D. 归并排序
11. 下列排序算法中, 在待排序数据已有序时, 花费时间反而最多的是()排序。
A. 冒泡 B. 希尔 C. 快速 D. 堆
12. 下列排序算法中, 在每一趟都能选出一个元素放到其最终位置上, 并且其时间性能受数据初始特性影响的是: ()。
A. 直接插入排序 B. 快速排序 C. 直接选择排序 D. 堆排序
13. 就平均性能而言, 目前最好的内排序方法是()排序法。
A. 冒泡 B. 希尔插入 C. 交换 D. 快速
14. 如果只想得到 1000 个元素组成的序列中第 5 个最小元素之前的部分排序的序列, 用 () 方法最快。
A. 起泡排序 B. 快速排列 C. Shell 排序 D. 堆排序 E. 简单选择排序
15. 在文件“局部有序”或文件长度较小的情况下, 最佳内部排序的方法是 ()
A. 直接插入排序 B. 冒泡排序 C. 简单选择排序
16. 下列排序算法中, () 算法可能会出现下面情况: 在最后一趟开始之前, 所有元素都不在其最终的位置上。
A. 堆排序 B. 冒泡排序 C. 快速排序 D. 插入排序
17. 下列排序算法中, 占用辅助空间最多的是: ()
A. 归并排序 B. 快速排序 C. 希尔排序 D. 堆排序
18. 从未排序序列中依次取出一个元素与已排序序列中的元素依次进行比较, 然后将其放在已排序序列的合适位置, 该排序方法称为()排序法。
A. 插入 B. 选择 C. 希尔 D. 二路归并
19. 在排序算法中, 每次从未排序的记录中挑出最小 (或最大) 关键码字的记录, 加入到已排序记录的末尾, 该排

- 序方法是 ()。
- A. 选择 B. 冒泡 C. 插入 D. 堆
20. 用直接插入排序方法对下面四个序列进行排序 (由小到大), 元素比较次数最少的是 ()。
- A. 94,32,40,90,80,46,21,69 B. 32,40,21,46,69,94,90,80
C. 21,32,46,40,80,69,90,94 D. 90,69,80,46,21,32,94,40
21. 直接插入排序在最好情况下的时间复杂度为 ()
- A. $O(\log n)$ B. $O(n)$ C. $O(n \cdot \log n)$ D. $O(n^2)$
22. 对序列{15, 9, 7, 8, 20, -1, 4, } 用希尔排序方法排序, 经一趟后序列变为{15, -1, 4, 8, 20, 9, 7}则该次采用的增量是 ()
- A. 1 B. 4 C. 3 D. 2
23. 对下列关键字序列用快速排序法进行排序时, 速度最快的情形是 ()。
- A.{21,25,5,17,9,23,30} B.{25,23,30,17,21,5,9} C.{21,9,17,30,25,23,5} D.{5,9,17,21,23,25,30}
24. 对关键字序列 28, 16, 32, 12, 60, 2, 5, 72 快速排序, 从小到大一次划分结果为 ()。
- A. (2,5,12,16)26(60,32,72) B. (5,16,2,12)28(60,32,72)
C. (2,16,12,5)28(60,32,72) D. (5,16,2,12)28(32,60,72)
25. 对 n 个记录的线性表进行快速排序为减少算法的递归深度, 以下叙述正确的是 ()
- A. 每次分区后, 先处理较短的部分 B. 每次分区后, 先处理较长的部分
C. 与算法每次分区后的处理顺序无关 D. 以上三者都不对
26. 在含有 n 个关键字的小根堆 (堆顶元素最小) 中, 关键字最大的记录有可能存储在 () 位置上。
- A. $\lfloor n/2 \rfloor$ B. $\lfloor n/2 \rfloor - 1$ C. 1 D. $\lfloor n/2 \rfloor + 2$
27. 下列四个序列中, 哪一个是堆 ()。
- A. 75,65,30,15,25,45,20,10 B. 75,65,45,10,30,25,20,15
C. 75,45,65,30,15,25,20,10 D. 75,45,65,10,25,30,20,15
28. 在对 n 个元素的序列进行排序时, 堆排序所需要的附加存储空间是 ()。
- A. $O(\log_2 n)$ B. $O(1)$ C. $O(n)$ D. $O(n \log_2 n)$
29. 对 n 个记录的文件进行堆排序, 最坏情况下的执行时间是多少? ()
- A. $O(\log_2 n)$ B. $O(n)$ C. $O(n \log_2 n)$ D. $O(n \cdot n)$
30. 有一组数据 (15, 9, 7, 8, 20, -1, 7, 4), 用堆排序的筛选方法建立的初始堆为 ()
- A. -1, 4, 8, 9, 20, 7, 15, 7 B. -1, 7, 15, 7, 4, 8, 20, 9
C. -1, 4, 7, 8, 20, 15, 7, 9 D. A, B, C 均不对。
31. 将两个各有 N 个元素的有序表归并成一个有序表,其最少的比较次数是()
- A. N B. $2N-1$ C. $2N$ D. $N-1$
32. 基于比较方法的 n 个数据的内部排序。最坏情况下的时间复杂度能达到的最好下界是 ()。
- A. $O(n \log n)$ B. $O(\log n)$ C. $O(n)$ D. $O(n \cdot n)$
33. 已知待排序的 n 个元素可分为 n/k 个组, 每个组包含 k 个元素, 且任一组内的各元素均分别大于前一组内的所有元素和小于后一组内的所有元素, 若采用基于比较的排序, 其时间下界应为。
- A. $O(n \log_2 n)$ B. $O(n \log_2 k)$ C. $O(k \log_2 n)$ D. $O(k \log_2 k)$

二、判断题:

1. 当待排序的元素很多时,为了交换元素的位置,移动元素要占用较多的时间,这是影响时间复杂度的主要因素。()
2. 内排序要求数据一定要以顺序方式存储。()
3. 排序算法中的比较次数与初始元素序列的排列无关。()
4. 排序的稳定性是指排序算法中的比较次数保持不变, 且算法能够终止。()
5. 在执行某个排序算法过程中, 出现了排序码朝着最终排序序列位置相反方向移动, 则该算法是不稳定的。()
6. 直接选择排序算法在最好情况下的时间复杂度为 $O(N)$ 。()
7. 在初始数据表已经有序时, 快速排序算法的时间复杂度为 $O(n \log_2 n)$ 。()
8. 在待排数据基本有序的情况下, 快速排序效果最好。()

9. 快速排序的速度在所有排序方法中为最快,而且所需附加空间也最少。()
10. 堆是满二叉树。()
11. (101, 88, 46, 70, 34, 39, 45, 58, 66, 10) 是堆。()
12. 在用堆排序算法排序时, 如果要进行增序排序, 则需要采用“大根堆”。()
13. 堆排序是稳定的排序方法。()
14. 归并排序辅助存储为 $O(1)$ 。()
15. 在分配排序时, 最高位优先分配法比最低位优先分配法简单。()
16. 冒泡排序和快速排序都是基于交换两个逆序元素的排序方法,冒泡排序算法的最坏时间复杂性是 $O(n*n)$,而快速排序算法的最坏时间复杂性是 $O(n\log_2^n)$,所以快速排序比冒泡排序算法效率更高。()
17. 快速排序和归并排序在最坏情况下的比较次数都是 $O(n\log_2 n)$ 。()
18. 归并排序在任何情况下都比所有简单排序速度快。()
19. 中序周游 (遍历) 平衡的二叉排序树, 可得到最后排序的关键码序列。()
20. 外部排序是把外存文件调入内存,可利用内部排序的方法进行排序,因此排序所花的时间取决于内部排序的时间。
21. 在外部排序时, 利用选择树方法在能容纳 m 个记录的内存缓冲区中产生的初始归并段的平均长度为 $2m$ 个记录。()
22. 影响外排序的时间因素主要是内存与外设交换信息的总次数。()

三、填空题

1. 若不考虑基数排序, 则在排序过程中, 主要进行的两种基本操作是关键字的_____和记录的_____。
2. 分别采用堆排序, 快速排序, 冒泡排序和归并排序, 对初态为有序的表, 则最省时间的是_____算法, 最费时间的是_____算法。
3. 对 n 个记录的表 $r[1..n]$ 进行简单选择排序, 所需进行的关键字间的比较次数为_____。
4. 设用希尔排序对数组{98, 36, -9, 0, 47, 23, 1, 8, 10, 7}进行排序, 给出的步长 (也称增量序列) 依次是 4, 2, 1 则排序需_____趟, 写出第一趟结束后, 数组中数据的排列次序_____。
5. 从平均时间性能而言, _____排序最佳。
6. 对于 7 个元素的集合{1, 2, 3, 4, 5, 6, 7}进行快速排序, 具有最小比较和交换次数的初始排列次序为_____。
7. 在数据表有序时, 快速排序算法的时间复杂度是_____。
8. 外部排序的基本方法是归并排序, 但在之前必须先生成_____。

四、应用题

1. 设待排序的关键码分别为 28, 13, 72, 85, 39, 41, 6, 20。按二分法插入排序算法已使前七个记录有序, 中间结果如下:

6	13	28	39	41	72	85	20
↑			↑			↑	
$i=1$			$m=4$			$r=7$	

试在此基础上, 沿用上述表达方式, 给出继续采用二分法插入第八个记录的比较过程。

- (1) 使用二分法插入排序所要进行的比较次数, 是否与待排序的记录的初始状态有关?
- (2) 在一些特殊情况下, 二分法插入排序比直接插入排序要执行更多的比较。这句话对吗?
2. 算法模拟

设待排序的记录共 7 个, 排序码分别为 8, 3, 2, 5, 9, 1, 6。

 - (1) 用直接插入排序。试以排序码序列的变化描述形式说明排序全过程 (动态过程) 要求按递减顺序排序。
 - (2) 用直接选择排序。试以排序码序列的变化描述形式说明排序全过程 (动态过程) 要求按递减顺序排序。
 - (3) 直接插入排序算法和直接选择排序算法的稳定性如何?
3. 我们知道, 对于 n 个元素组成的线性表进行快速排序时, 所需进行的比较次数与这 n 个元素的初始排序有关。问:
 - (1) 当 $n=7$ 时, 在最好情况下需进行多少次比较? 请说明理由。
 - (2) 当 $n=7$ 时, 给出一个最好情况的初始排序的实例。

- (3) 当 $n=7$ 时, 在最坏情况下需进行多少次比较? 请说明理由。
- (4) 当 $n=7$ 时, 给出一个最坏情况的初始排序的实例。
4. 全国有 10000 人参加物理竞赛, 只录取成绩优异的前 10 名, 并将他们从高分到低分输出。而对落选的其他考生, 不需排出名次, 问此种情况下, 用何种排序方法速度最快? 为什么?
5. 已知待排序的序列为 (503, 87, 512, 61, 908, 170, 897, 275, 653, 462), 试完成下列各题。
- (1) 根据以上序列建立一个堆 (画出第一步和最后堆的结果图), 希望先输出最小值。
- (2) 输出最小值后, 如何得到次小值。(并画出相应结果图)
6. (1) 判定起泡排序的结束条件是什么?
- (2) 请简单叙述希尔排序的基本思想。
- (3) 将下列序列调整成堆 (堆顶为最小值)
- | | | | | | | | | | |
|-----|----|----|----|----|----|----|----|----|----|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
| 112 | 70 | 33 | 65 | 24 | 56 | 48 | 92 | 80 | 13 |
- (4) 在 16 个关键字中选出最小的关键字至少要多少次比较? 再选出次小的关键字至少要多少次比较? 请简要说明选择的方法和过程。
7. 奇偶交换排序如下所述: 对于初始序列 $A[1], A[2], \dots, A[n]$, 第一趟对所有奇数 i ($1 \leq i < n$), 将 $A[i]$ 和 $A[i+1]$ 进行比较, 若 $A[i] > A[i+1]$, 则将两者交换; 第二趟对所有偶数 i ($2 \leq i < n$), 将 $A[i]$ 和 $A[i+1]$ 进行比较, 若 $A[i] > A[i+1]$, 则将两者交换; 第三趟对所有奇数 i ($1 \leq i < n$); 第四趟对所有偶数 i ($2 \leq i < n$), $\dots$, 依次类推直至到整个序列有序为止。
- (1) 分析这种排序方法的结束条件。
- (2) 写出用这种排序方法对 35, 70, 33, 65, 24, 21, 33 进行排序时, 每一趟的结果。

五、算法设计题:

1. 冒泡排序算法是把大的元素向上移 (气泡的上浮), 也可以把小的元素向下移 (气泡的下沉) 请给出上浮和下沉过程交替的冒泡排序算法。
2. 有一种简单的排序算法, 叫做计数排序 (count sorting)。这种排序算法对一个待排序的表 (用数组表示) 进行排序, 并将排序结果存放到另一个新的表中。必须注意的是, 表中所有待排序的关键码互不相同, 计数排序算法针对表中的每个记录, 扫描待排序的表一趟, 统计表中有多少个记录的关键码比该记录的关键码小, 假设针对某一个记录, 统计出的计数值为 c , 那么, 这个记录在新的有序表中的合适的存放位置即为 c 。
- (1) 给出适用于计数排序的数据表定义;
- (2) 使用 Pascal 或 C 语言编写实现计数排序的算法;
- (3) 对于有 n 个记录的表, 关键码比较次数是多少?
- (4) 与简单选择排序相比较, 这种方法是否更好? 为什么?
3. 设有顺序放置的 n 个桶, 每个桶中装有一粒砾石, 每粒砾石的颜色是红、白、蓝之一。要求重新安排这些砾石, 使得所有红色砾石在前, 所有白色砾石居中, 所有蓝色砾石居后, 重新安排时对每粒砾石的颜色只能看一次, 并且只允许交换操作来调整砾石的位置。

参考答案

一、选择题

1. D	2. D	3. C	4. A	5. A	6. C	7. C	8. B	9. C	10. D	11. C
12. B	13. D	14. D	15. A	16. D	17. A	18. A	19. A	20. C	21. B	22. B
23. A	24. B	25. A	26. D	27. C	28. B	29. C	30. C	31. A	32. A	33. B

二、判断题:

1. $\checkmark$	2. $\times$	3. $\times$	4. $\times$	5. $\times$	6. $\times$	7. $\times$	8. $\times$	9. $\times$	10. $\times$	11. $\checkmark$
12. $\checkmark$	13. $\times$	14. $\times$	15. $\times$	16. $\times$	17. $\times$	18. $\times$	19. $\checkmark$	20. $\times$	21. $\checkmark$	22. $\checkmark$

三、填空题

1. 比较, 移动	2. 冒泡, 快速	3. $n(n-1)/2$	4. 3, (10, 7, -9, 0, 47, 23, 1, 8, 98, 36)
5. 快速	6. (4, 1, 3, 2, 6, 5, 7)	7. $O(n^2)$	8. 初始归并段 (顺串)

四、应用题

1. 6, 13, 28, 39, 41, 72, 85, 20
 $i=1 \uparrow$ $m=4 \uparrow$ $r=7 \uparrow$

20 < 39 $i=1 \uparrow$ $m=2 \uparrow$ $r=3 \uparrow$

20 > 13 $i=3$ $r=3$ $m=3$

20 < 28 $r=2$ $i=3$

将 $r+1$ (即第 3 个) 后的元素向后移动, 并将 20 放入 $r+1$ 处, 结果为 6,13,20,28,39,41,72,85。

(1) 使用二分法插入排序所要进行的比较次数, 与待排序的记录初始状态无关。不论待排序序列是否有序, 已形成的部分子序列是有序的。二分法插入首先查找插入位置, 插入位置是判定树查找失败的位置。失败位置只能在判定树的最下两层上。

(2) 一些特殊情况下, 二分法插入排序要比直接插入排序要执行更多的比较。例如, 在待排序序列已有序的情况下就是如此。

2. (1) 直接插入排序

第一趟 (3)[8,3],2,5,9,1,6 第二趟 (2)[8,3,2],5,9,1,6 三趟 (5)[8,5,3,2],9,1,6

第四趟 (9)[9,8,5,3,2],1,6 第五趟 (1)[9,8,5,3,2,1],6 第六趟 (6)[9,8,6,5,3,2,1]

(2) 直接选择排序 (第六趟后仅剩一个元素, 是最小的, 直接选择排序结束)

第一趟 (9)[9],3,2,5,8,1,6 第二趟 (8)[9,8],2,5,3,1,6 第三趟 (6)[9,8,6],5,3,1,2

第四趟 (5)[9,8,6,5],3,1,2 第五趟 (3)[9,8,6,5,3],1,2 第六趟 (2)[9,8,6,5,3,2],1

(3) 直接插入排序是稳定排序, 直接选择排序是不稳定排序。

3. (1) 在最好情况下, 假设每次划分能得到两个长度相等的子文件, 文件的长度 $n=2^k-1$, 那么第一遍划分得到两个长度均为 $\lfloor n/2 \rfloor$ 的子文件, 第二遍划分得到 4 个长度均为 $\lfloor n/4 \rfloor$ 的子文件, 以此类推, 总共进行 $k=\log_2(n+1)$ 遍划分, 各子文件的长度均为 1, 排序完毕。当 $n=7$ 时, $k=3$, 在最好情况下, 第一遍需比较 6 次, 第二遍分别对两个子文件 (长度均为 3, $k=2$) 进行排序, 各需 2 次, 共 10 次即可。

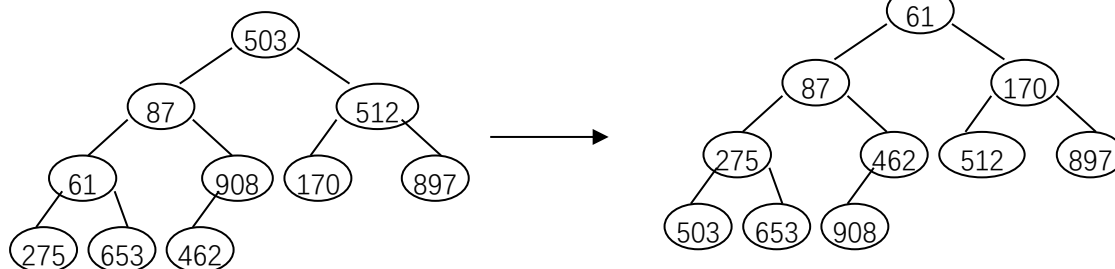
(2) 在最好情况下快速排序的原始序列实例: 4,1,3,2,6,5,7。

(3) 在最坏情况下, 若每次用来划分的记录的关键字具有最大值 (或最小值), 那么只能得到左 (或右) 子文件, 其长度比原长度少 1。因此, 若原文件中的记录按关键字递减次序排列, 而要求排序后按递增次序排列时, 快速排序的效率与冒泡排序相同, 其时间复杂度为 $O(n^2)$ 。所以当 $n=7$ 时, 最坏情况下的比较次数为 21 次。

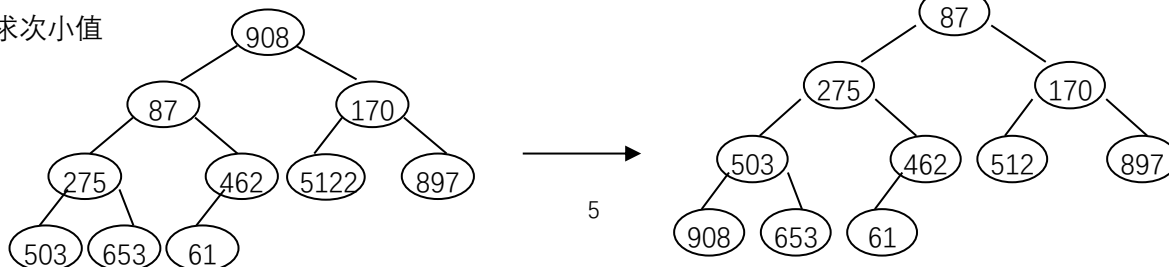
(4) 在最坏情况下快速排序的初始序列实例: 7,6,5,4,3,2,1, 要求按递增排序。

4. 在内部排序方法中, 一趟排序后只有简单选择排序和冒泡排序可以选出一个最大 (或最小) 元素, 并加入到已有的有序子序列中, 但要比较 $n-1$ 次。选次大元素要再比较 $n-2$ 次, 其时间复杂度是 $O(n^2)$ 。从 10000 个元素中选 10 个元素不能使用这种方法。而快速排序、插入排序、归并排序、基数排序等时间性能好的排序, 都要等到最后才能确定各元素位置。只有堆排序, 在未结束全部排序前, 可以有部分排序结果。建立堆后, 堆顶元素就是最大 (或最小, 视大堆或小堆而定) 元素, 然后, 调堆又选出次大 (小) 元素。凡要求在 n 个元素中选出 k ($k < n, k > 2$) 个最大 (或最小) 元素, 一般均使用堆排序。因为堆排序建堆比较次数至多不超过 $4n$, 对深度为 k 的堆, 在调堆算法中进行的关键词的比较次数至多为 $2(k-1)$ 次, 且辅助空间为 $O(1)$ 。

5. (1) 建小堆



(2) 求次小值



6. (1)当至多进行 $n-1$ 趟起泡排序, 或一趟起泡排序中未发生交换(即已有序)时, 结束排序。

(2)希尔排序是对直接插入排序算法的改进, 它从“记录个数少”和“基本有序”出发, 将待排序的记录划分成几组(缩小增量分组), 从而减少参与直接插入排序的数据量, 当经过几次分组排序后, 记录的排列已经基本有序, 这个时候再对所有的记录实施直接插入排序。

(3)13, 24, 33, 65, 70, 56, 48, 92, 80, 112

(4)采用树型锦标赛排序选出最小关键字至少要 15 次比较。再选出次小关键字比较 4 次。(两两比较 8 次选出 8 个优胜, 再两两比较 4 次选出 4 个优胜, 半决赛两场, 决赛一场, 共比较了 15 次。将冠军的叶子结点改为最大值, 均与兄弟比较, 共 4 次选出亚军。)

7. (1)排序结束条件为没有交换为止

第一趟奇数: 35,70,33,65,21,24,33 第二趟偶数: 35,33,70,21,65,24,33

第三趟奇数: 33,35,21,70,24,65,33 第四趟偶数: 33,21,35,24,70,33,65

第五趟奇数: 21,33,24,35,33,70,65 第六趟偶数: 21,24,33,33,35,65,70

第七趟奇数: 21,24,33,33,35,65,70(无交换) 第八趟偶数: 21,24,33,33,35,65,70(无交换) 结束

五、算法设计题:

1. **void** BubbleSort2(**int** a[],**int** n) //相邻两趟向相反方向起泡的冒泡排序算法

```
{ change=1;low=0;high=n-1; //冒泡的上下界
  while(low<high && change)
  { change=0;                //设不发生交换
    for(i=low;i<high;i++) //从上向下起泡
      if(a[i]>a[i+1]){a[i]<-->a[i+1];change=1;} //有交换, 修改标志 change
    high--; //修改上界
    for(i=high;i>low;i--) //从下向上起泡
      if(a[i]<a[i-1]){a[i]<-->a[i-1];change=1;}
    low++; //修改下界      }//while
  }//BubbleSort2
```

[算法讨论]题目中“向上移”理解为向序列的右端, 而“向下移”按向序列的左端来处理。

2. **typedef struct**

```
{ int key; datatype info;}RecType
void CountSort(RecType a[],b[],int n) //计数排序算法, 将 a 中记录排序放入 b 中
{ for(i=0;i<n;i++) //对每一个元素
  { for(j=0,cnt=0;j<n;j++)
    if(a[j].key<a[i].key) cnt++; //统计关键字比它小的元素个数
    b[cnt]=a[i]; }
  }//Count_Sort
```

(3) 对于有 n 个记录的表, 关键码比较 n^2 次。

(4) 简单选择排序算法比本算法好。简单选择排序比较次数是 $n(n-1)/2$, 且只用一个交换记录的空间; 而这种方法比较次数是 n^2 , 且需要另一数组空间。

[算法讨论]因题目要求“针对表中的每个记录, 扫描待排序的表一趟”, 所以比较次数是 n^2 次。若限制“对任意两个记录之间应该只进行一次比较”, 则可把以上算法中的比较语句改为:

```
for(i=0;i<n;i++) a[i].count=0; //各元素再增加一个计数域, 初始化为 0
for(i=0;i<n;i++)
  for(j=i+1;j<n;j++)
```

```
if(a[i].key<a[j].key) a[j].count++; else a[i].count++;
```

3. [题目分析]利用快速排序思想解决。由于要求“对每粒砾石的颜色只能看一次”, 设 3 个指针 i , j 和 k , 分别

指向红色、白色砾石的后一位置和待处理的当前元素。从 $k=n$ 开始，从右向左搜索，若该元素是兰色，则元素不动，指针左移（即 $k-1$ ）；若当前元素是红色砾石，分 $i \geq j$ （这时尚未有白色砾石）和 $i < j$ 两种情况。前一情况执行第 i 个元素和第 k 个元素交换，之后 $i+1$ ；后一情况， i 所指的元素已处理过（白色）， j 所指的元素尚未处理，应先将 i 和 j 所指元素交换，再将 i 和 k 所指元素交换。对当前元素是白色砾石的情况，也可类似处理。

为方便处理，将三种砾石的颜色用整数 1、2 和 3 表示。

```
void QkSort(rectype r[],int n)
// r 为含有 n 个元素的线性表，元素是具有红、白和兰色的砾石，用顺序存储结构存储，
// 本算法对其排序，使所有红色砾石在前，白色居中，兰色在最后。
{int i=1,j=1,k=n,temp;
 while (k!=j)
 {while (r[k].key==3) k--; // 当前元素是兰色砾石，指针左移
  if (r[k].key==1) // 当前元素是红色砾石
   if (i>=j){temp=r[k];r[k]=r[i];r[i]=temp; i++;}
   //左侧只有红色砾石，交换 r[k]和 r[i]
  else {temp=r[j];r[j]=r[i];r[i]=temp; j++;
   //左侧已有红色和白色砾石，先交换白色砾石到位
   temp=r[k];r[k]=r[j];r[j]=temp; i++;
   //白色砾石（i 所指）和待定砾石（j 所指）
   } //再交换 r[k]和 r[i]，使红色砾石入位。
  if (r[k].key==2)
   if (i<=j) { temp=r[k];r[k]=r[j];r[j]=temp; j++;}
   // 左侧已有白色砾石，交换 r[k]和 r[j]
  else { temp=r[k];r[k]=r[i];r[i]=temp; j=i+1;}
   //i、j 分别指向红、白色砾石的后一位置
 }//while
 if (r[k]==2) j++; /* 处理最后一粒砾石
 else if (r[k]==1) { temp=r[j];r[j]=r[i];r[i]=temp; i++; j++;}
 //最后红、白、兰色砾石的个数分别为: i-1;j-i;n-j+1
 }//结束 QkSor 算法
```

[算法讨论]若将 j （上面指向白色）看作工作指针，将 $r[1..j-1]$ 作为红色， $r[j..k-1]$ 为白色， $r[k..n]$ 为兰色。从 $j=1$ 开始查看，若 $r[j]$ 为白色，则 $j=j+1$ ；若 $r[j]$ 为红色，则交换 $r[j]$ 与 $r[i]$ ，且 $j=j+1$ ， $i=i+1$ ；若 $r[j]$ 为兰色，则交换 $r[j]$ 与 $r[k]$ ， $k=k-1$ 。算法进行到 $j>k$ 为止。

算法片段如下：

```
int i=1,j=1,k=n;
while(j<=k)
 if (r[j]==1) //当前元素是红色
  {temp=r[i]; r[i]=r[j]; r[j]=temp; i++;j++;}
 else if (r[j]==2) j++; //当前元素是白色
 else //r[j]==3 当前元素是兰色
  {temp=r[j]; r[j]=r[k]; r[k]=temp; k--;}

```

对比两种算法，可以看出，正确选择变量（指针）的重要性。